

**Master d'Informatique -
Module MAOA**

Recherche Opérationnelle
et Optimisation Combinatoire

Ordonnancement et Programmation
Mathématique

Pierre Fouilhoux

2024-2025

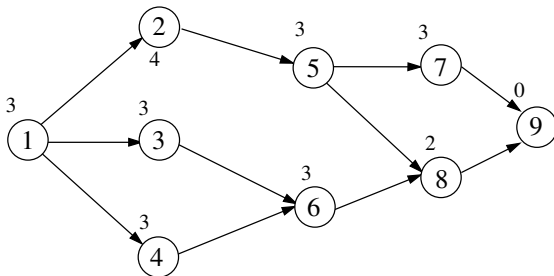
1. Problème central $m \mid prec \mid C_{max}$
2. Dimensionnement de ressource en ordonnancement
3. Resource Constrained Project Scheduling Projet : RCPSP
4. Scheduling under uncertainty
5. Supply Chain par le problème de Production-Routing

Problème central

On considère le problème d'ordonnancement classique $n \mid prec \mid C_{max}$.

- $V = \{1, \dots, n\}$ un ensemble de n tâches associées respectivement à des temps d'exécution $p_i, i \in V$.
- $G = (V, A)$ un graphe ordonné **acircuitique** dit de précédence : chaque arc (i, j) de G est une contrainte de précédence telle que la tâche i doit être achevée avant le début de la tâche j .
- une tâche $s \in V$ appelée source qui n'a pas de prédecesseur et peut être effectuée en premier ; et une tâche t appelée puits qui n'a pas de successeur, qui a une durée d'exécution nulle $p_t = 0$ et qui peut être effectuée en dernier.

Exemple : avec $n = 9$ tâches avec $s = 1$ et $t = 9$; les p_i sont à coté du sommet-tâche.



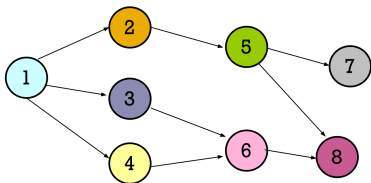
Cas concret

Tâche	Description	Durée (semaines)	Prédécesseur(s)
J_1	terrassement	2	-
J_2	fondations	2	J_1
J_3	canalisations enterrées	1	J_2
J_4	dalle du plancher	3	J_3
J_5	murs porteurs	3	J_4
J_6	portes et fenêtres extérieures	1	J_5
J_7	charpente	1	J_5
J_8	couverture	2	J_7
J_9	isolation	1	J_6, J_8
J_{10}	cloisons et portes	1	J_6, J_8
J_{11}	revêtements extérieurs et volets	2	J_6, J_8
J_{12}	electricité	1	J_9, J_{10}
J_{13}	plomberie	2	J_9, J_{10}
J_{14}	revêtements sols et murs	2	J_{12}, J_{13}
J_{15}	appareillages et équipements	1	J_{14}

Ordonnancement

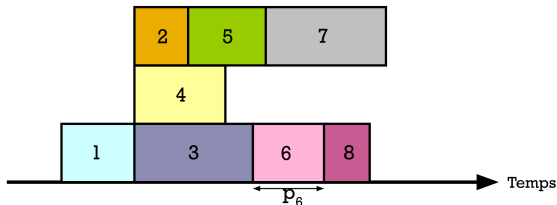
Output : la solution du problème est un *ordonnancement* : l'ensemble des dates x_i de début de la tâche i pour tout $i \in V$.

Objectif : déterminer la date de fin de la tâche x_t soit la plus petite possible pour un ordonnancement : cette date de la tâche t s'appelle alors le *makespan*.



Une solution admissible peut être trouvée par un simple parcours en largeur donnant une liste topologique

$p_1=3$
 $p_2=2$
 $p_3=6$
 $p_4=5$
 $p_5=4$
 $p_6=3$
 $p_7=6$
 $p_8=2$



Méthodes algorithmiques

Méthode potentiel-tâches : graphe où chaque tâche est représentée par 2 sommets. Elle est moins intéressante que la méthode PERT.

Méthode PERT : à partir du graphe de précédence G :

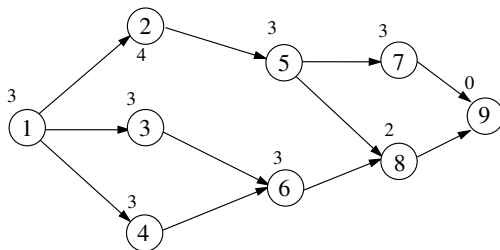
- Valuer chaque arc issu du sommet i à la valuation p_i
- Le problème revient à déterminer un plus long chemin de s à t dans ce graphe.

En effet, le plus long chemin entre s et t va correspondre à la plus longue séquence de tâches qui ne doivent être faites les unes après les autres. La longueur de ce chemin est bien la date x_t d'un ordonnancement possible : il s'agit bien du makespan.

Comme G est acyclique, le problème est polynomial !
(par une adaption des algos de plus courts chemins classiques).

Méthodes algorithmiques

Pour



le plus long chemin est donné par la séquence des tâches 1, 2, 5, 7, 9 est vaut 13 :
c'est la valeur du makespan C_{max} .

Analyse de la solution

Autres informations :

- Tout plus long chemin de s à t est appelé *chemin critique* : car tout allongement d'une durée d'une tâche le long de ce chemin va allonger le makespan.
- En calculant l'arbre des plus longs chemins à partir de s , on fixe une date "au plus tôt" x_i pour chaque tâche.
- Une tâche qui n'est pas sur un chemin critique peut s'allonger "un peu" ou être retardée sans perturber le makespan
- Si on connaît une date D_t (avec $D_t > C_{max}$) à laquelle la tâche t ne peut pas être repoussée (une date maximale de fin chantier par exemple) : on peut calculer le plus long chemin dans le graphe G^{-1} obtenu en retournant tous les arcs. Cette valeur donne la date de démarrage au plus tard de l'ordonnancement pour que D_t soit respecté.
- En calculant l'arbre des plus longs chemins entre t et s dans G^{-1} , on a i la date de démarrage au plus tard de chaque tâche sans retarder la fin D_t de l'ordonnancement
- Ainsi, pour toute tâche i $[x_i, D_i]$ est l'intervalle dans le lequel on peut déplacer la date de la tâche en respectant la date D_t de fin. Si $D_t = C_{max}$, ces *intervalles de liberté* représentent toutes les solutions optimales du problème.

Première formulation

On s'intéresse à donner des formulations par des Programmes Linéaires (en variables continues ou entières) pour ce problème.

Première formulation

On considère des variables réelles x_i associée à chaque tâche de V . Elles vont représenter la date de début de tâche dans un ordonnancement dans le problème (P) :

$$\text{Min } x_t$$

$$x_i + p_i \leq x_j \quad \forall (i, j) \in A \quad (1)$$

$$x_i \geq 0 \quad \forall j \in V \quad (2)$$

On peut voir que, pour une solution optimale, x_t^* est bien le makespan.

En effet, x^* donne des valeurs respectant les inégalités de précédence en minimisant x_t .

Les valeurs x_j peuvent être n'importe quelle valeur dans l'intervalle de liberté.

Deuxième formulation

Deuxième formulation

On considère à présent des variables réelles y_{ij} pour tout arc $(i, j) \in A$.
Voici une deuxième formulation (D).

$$\text{Max} \sum_{(i,j) \in A} p_i y_{ij}$$

$$\sum_{(s,j) \in A} y_{sj} \leq 1 \quad (3)$$

$$\sum_{(j,t) \in A} y_{jt} \leq 1 \quad (4)$$

$$\sum_{(i,j) \in A} y_{ij} = \sum_{(j,k) \in A} y_{jk} \quad \forall j \in V \setminus \{s, t\} \quad (5)$$

$$y_{ij} \geq 0 \quad \forall (i, j) \in A \quad (6)$$

C'est une formulation “de flot” où un flot de valeur 1 va circuler de s à t) sur un unique chemin de coût minium. Et ce coût est la valeur du plus long chemin !

Deuxième formulation

Lemma

Une solution optimale entière à ce programme linéaire (D) donne la solution du problème.

Preuve : Il y a deux cas :

- soit tous les arcs sortant de s sont nuls et tout le flot est nul : ce n'est pas une solution optimale
- soit il y a un arc unique entier sortant de s dans la solution. Par les inégalités de conservation du flot, à chaque de valeur 1 entrant dans un sommet, un unique arc de valeur sort de ce sommet : ainsi cela désigne une suite d'arc qui aura pour dernier sommet le puits t . Cela désigne donc un chemin de s à t .
- Par la fonction objective, il va s'agir d'un plus long chemin. □

Deuxième formulation

Ecrivons (D) ainsi :

$$\begin{aligned} & \text{Max} \sum_{(i,j) \in A} p_i y_{ij} \\ & - \sum_{(s,j) \in A} y_{sj} \geq -1 \end{aligned} \quad (7)$$

$$+ \sum_{(j,t) \in A} y_{jt} \leq 1 \quad (8)$$

$$- \sum_{(i,j) \in A} y_{ij} + \sum_{(j,k) \in A} y_{jk} = 0 \quad \forall j \in V \setminus \{s, t\} \quad (9)$$

$$y_{ij} \geq 0 \quad \forall (i,j) \in A \quad (10)$$

Lemma

Toute solution réelle optimale de (D) est entière

Preuve : C'est une formulation de flot : la matrice est bien TU par le thm de Poincaré : en effet, chaque variable a exactement un $+1$ et -1 par colonne. $\square$

Donc toutes solutions optimales y^* de (D) est entière.

Relation primale-duale

Le dual (P') de (D) peut être écrit ainsi :

On associe à chaque ligne de (D) une variable x_i pour chaque $i \in V$.

$$\begin{aligned} \text{Min} \quad & x_t - x_s \\ & x_i + p_i \leq x_j \quad \forall (i, j) \in A \end{aligned} \tag{11}$$

$$x_s \leq 0 \tag{12}$$

$$x_t \geq 0 \tag{13}$$

$$x_i \in \mathbf{R} \quad \forall j \in V \setminus \{s, t\} \tag{14}$$

On retrouve “presque” le problème (P) . Une solution optimale de ce problème va consister à prendre $x_s = 0$ et $x_t \geq 0$ le plus petit possible donc même si les variables x_i sont libres dans $\mathbf{R}$, elles sont en fait positives : il s’agit d’une écriture qui coïncide en 0 avec (P) .

Relation primale-duale

Lemma

Le système formé par (D) et (P') est TDI.

Preuve : Soit un vecteur de coût entiers c_{ij} quelconque associé aux arcs de G .

Le dual du problème a bien la même forme avec des inégalités $x_i + c_{ij} \leq x_j \quad \forall (i, j) \in A$.

Pour une solution optimale de (D) qui est donc un plus long chemin de s à t selon le poids c , on peut faire coïncider une solution entière x de (P') en fixant $x_s = 0$.

x_i égal au coût du plus long chemin de s à i dans G .

Les deux solutions ont bien même valeur.

Donc le système est bien TDI. □

La conséquence, que l'on savait déjà par le caractère TU, est que (D) est un polytope entier : donc que la solution y^* de (D) est entière.

Le problème de recherche d'un plus long chemin et celui de l'ordonnancement du problème central sont duaux !

1. Problème central $m \mid prec \mid C_{max}$
2. Dimensionnement de ressource en ordonnancement
3. Resource Constrained Project Scheduling Projet : RCPSP
4. Scheduling under uncertainty
5. Supply Chain par le problème de Production-Routing

Poset

Un *ensemble partiellement ordonné* (poset) est un couple $(V, \prec)$ où

- $V = \{1, \dots, n\}$ est un ensemble d'éléments
- et $\prec$ une relation d'ordre partiel.

On dit que si $i \prec j$, alors *i est avant j* .

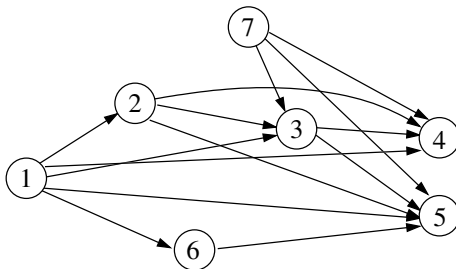
Une relation d'ordre partiel est une relation binaire entre les éléments de V telle que, pour tout triplet i, j, k d'éléments distincts de V , si $i \prec j$ et $j \prec k$ alors $i \prec k$.

Noter que l'on a donc trois cas distincts : soit $i \prec j$; soit $j \prec i$; soit i et j sont incomparables selon $\prec$.

Représentation graphique

On peut représenter un poset par un graphe orienté $G_{\prec} = (V, A)$ défini de la façon suivante :

- V est l'ensemble d'éléments du poset $(V, \prec)$
- A est un ensemble d'arcs tel qu'il existe un arc (i, j) dans A si et seulement si $i \prec j$.



Antichaîne

On appelle *antichaîne* un ensemble d'éléments deux à deux incomparables dans le poset $(V, \prec)$.

Dans l'exemple du poset Po_1 , $\{2, 6, 7\}$ et $\{3, 6\}$ sont des antichaînes, mais pas $\{2, 5, 7\}$ car $2 \prec 5$.

Etant donné un poids w_i associé à chaque élément de V , on note alors $w(A) = \sum_{i \in A} w_i$ le poids de A .

On appelle *problème de l'antichaîne de poids maximum* (PAM) le problème consistant à rechercher une antichaîne de $(V, \prec)$ de poids maximal.

Ressource

On considère :

- un ensemble de tâche $T = \{1, \dots, n\}$ à réaliser.
- et une relation de précédence sur les tâches de manière à ce que, pour deux tâches i, j de V , si i est avant j alors la tâche i doit avoir été réalisée avant que débute la tâche j .
- Pour être réalisée, une tâche $i \in V$ utilise une quantité $r_i \in \mathbf{R}$ de *ressource* pour être réalisée (cette ressource peut par exemple être un nombre d'ouvriers, une consommation électriques,...). Ainsi, si deux tâches i et j ont lieu en même temps, alors la quantité de ressource utilisée à ce moment est $r_i + r_j$.

Question Déterminer la quantité maximale de ressource qui peut être utilisée simultanément pendant l'exécution d'un ordonnancement.

Ressources et antichaînes

Lemma

La question de dimensionnement des ressources se ramène au problème problème de l'antichaîne maximale

Preuve :

En effet, On ne connaît pas à l'avance quelles sont les tâches qui vont être effectuées simultanément ou non : en effet, on ne connaît pas à l'avance la durée de ces tâches, ni leur date d'exécution. La quantité maximale de ressource utilisable correspond à un ensemble de tâches pouvant être réalisées simultanément. Or les antichaînes sont exactement les ensembles de tâches pouvant être réalisées simultanément. Donc déterminer la quantité maximale de ressource utilisable revient à rechercher une antichaîne de plus grande somme des ressources. □

Formulation PLNE

Soit un poset $(V, \prec)$ où $V = \{1, \dots, n\}$ donné par son graphe $G_{\prec}$ et un poids w_i associé aux éléments de V . On considère une variable binaire x_i associée à chaque élément i de V .

La formulation suivante est une formulation F du PAM.

$$\text{Max} \sum_{i \in V} w_i x_i$$

$$x_i + x_j \leq 1 \quad \forall i, j \in V \text{ avec } i \prec j, \quad (15)$$

$$x_i \leq 1 \quad \forall i \in V, \quad (16)$$

$$x_i \geq 0 \quad \forall i \in V, \quad (17)$$

$$x_i \in \mathbf{N} \quad \forall i \in V.$$

On peut remarquer que toute solution de la formulation correspond à des ensembles de tâches sans lien de précédence deux à deux : il s'agit donc d'une antichaîne. D'autre part, toute antichaîne correspond à une solution de la formulation. Le PLNE est bien équivalent à PAM.

La formulation est compacte et peut donc être résolu par un solveur PLNE (Branch-and-Bound avec génération automatique de contraintes).

Complexité du problème de l'antichaîne

On a ramené ce problème à un PLNE...

Complexité du problème de l'antichaîne

On a ramené ce problème à un PLNE...

C'est une erreur ! Ce problème est polynomial !

C'est une méthode un peu complexe (dont le moteur est le théorème de Dilworth plus l'algorithme de Koning)...

Mais peut-on le résoudre par la PL ?

Structure combinatoire

Soit un poset $(V, \prec)$ et son graphe $G_{\prec}$.

- Si $G_{\prec}$ contient un chemin μ reliant un sommet i à un sommet j , alors $G_{\prec}$ contient l'arc (i, j) .

En effet : Notons $i_0 = i, i_1, \dots, i_k = j$ les sommets du chemin μ . Par définition, comme $i_l \prec i_{l+1}$ et $i_{l+1} \prec i_{l+2}$ alors $i_l \prec i_{l+2}$. Donc il existe un chemin μ' passant reliant les sommets de μ en sautant i_{l+1} . En réitérant ce processus, on arrive à un chemin reliant directement i à j , c'est-à-dire l'arc (i, j) .

- Le graphe $G_{\prec}$ d'un poset ne contient pas de circuit.

Si $G_{\prec}$ contenait un circuit passant par i, j, k , alors d'après la propriété précédent, on aurait un circuit $(i \prec j, j \prec k, k \prec i)$ ce qui contredit la définition car on devrait avoir $i \prec k$.

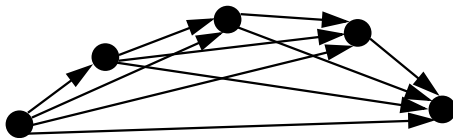
P-chaîne

Un ensemble $C \subset V$ est appelé *P-chaîne* de $(V, \prec)$ si $i \prec j$ ou $j \prec i$ pour tout $i, j \in C$.

Remarquons que tout élément de V constitue une P-chaîne et que, pour i, j dans V , $\{i, j\}$ est une P-chaîne si et seulement si $i \prec j$.

Lemma

La représentation graphique d'une P-chaîne correspond aux graphes de la structure suivante (ici donné pour une P-chaîne de 5 éléments).



Preuve : Une P-chaîne à 1 sommet et un sommet isolé. (Remarque inutile : une P-chaîne à deux sommets est un arc.) Supposons qu'une P-chaîne à p sommets respecte cette structure. Considérons une P-chaîne C à $p + 1$ éléments. On peut prouver qu'il existe un élément i qui est avant tous les autres. En effet, si ce n'était pas le cas, le graphe de cette P-chaîne contiendrait un circuit. Ainsi $C \setminus \{i\}$ est une P-chaîne à n élément. Donc par HR, elle a la structure du dessin. Comme i est avant tous les autres, on obtient bien le même dessin.

P-chaîne et points extrêmes

Soit une P-chaîne C du poset.

Par le lemme précédent, on peut décrire C par ses k éléments $i_1, \dots, i_k$, numérotés dans l'ordre donné par la structure combinatoire de la question précédente, c'est-à-dire $i_l \prec i_{l+t}$ pour tout $l \in \{1, \dots, k-1\}$ et pour tout $t \in \{1, \dots, k-l\}$.

Lemma

Le point y^ défini de la façon suivante :*

$y_i^ = \frac{1}{2}$ pour tout $i \in C$ et 0 sinon. est un point extrême de la relaxation linéaire du PLNE.*

Preuve : Clairement y^* vérifie bien les inégalités triviales. De plus, si on a i et j tel que $i \prec j$ alors dans le pire des cas $y_i^* + y_j^* = 1$, donc les contraintes sont bien toutes vérifiées.

Pour les $n - |C|$ éléments en dehors de la P-chaîne, y^* vérifie $x_i \geq 0$ à l'égalité. De plus, pour tout i, j de la P-chaîne, on a $i \prec j$ ou $j \prec i$ donc la contrainte $x_i + x_j \leq 1$ est satisfaite à l'égalité par y^* , on a donc $\frac{|C|(|C|-1)}{2}$ inégalités ici. Parmi toutes ces inégalités, on veut en déterminer $|C|$ linéairement indépendantes. Pour cela, notons i_1 l'élément de la chaîne telle que $i_0 \prec j$ pour tout élément $C \setminus \{i_0\}$ et i_2, i_3 deux éléments distincts de i_0 de la P-chaîne : alors les inégalités $x_{i_1} + x_j \leq 1$ pour tout $j \in V \setminus \{i_0\}$ et $x_{i_2} + x_{i_3} \leq 1$ sont linéairement indépendantes. Donc y^* satisfait bien n inégalités linéairement indépendantes, il s'agit bien d'un point extrême de la formulation. $\square$

Inégalités de P-chaîne

Donc la relaxation linéaire du PLNE n'est pas un polyèdre entier...

Mes inégalités

$$\sum_{i \in C} x_i \leq 1 \text{ pour toute P-chaîne } C \quad (18)$$

sont valides pour la formulation.

On les appelle des *inégalités de P-chaînes*.

En effet, Soit x une solution correspondant à une antichaîne. Dans une P-chaîne C , une antichaîne ne peut pas contenir plus d'un élément. Donc la contrainte est valide.

Remarque : ces inégalités coupent le point y^* .

Nouvelle formulation

Considérons un poset $(V, \prec)$ et un poids $w_i \in \mathbf{R}$, associé à chaque élément $i \in V$. On considère le programme linéaire (P) suivant :

$$\begin{aligned} \text{Max } & \sum_{i \in V} w_i x_i \\ & \sum_{i \in C} x_i \leq 1 \quad \text{pour toute P-chaîne } C, \\ & x_i \geq 0 \quad \text{pour tout } i \in V. \end{aligned}$$

Theorem

Les inégalités de P-chaîne et les inégalités triviales sont une caractérisation complète du polyèdre des antichaînes.

i.e. ce PL est un polyèdre entier dont les solutions sont les antichaînes.

Caractérisation du polyèdre

Preuve : Une preuve de ce résultat peut-être obtenue en prouvant le caractère TDI de cette formulation linéaire et grâce au théorème de Dilworth.

Considérons l'ensemble $\mathcal{C}$ de toutes les P-chaînes de $(V, \prec)$. On associe la variable duale λ_C à l'inégalité de P-chaîne associée à la P-chaîne C de $\mathcal{C}$. Le programme linéaire (D) suivant est le dual de (P) .

$$\begin{aligned} \text{Min } & \sum_{C \in \mathcal{C}} \lambda_C \\ & \sum_{i \in V} \lambda_C \geq w_i \quad \text{pour tout } i \in V, \\ & \lambda_C \geq 0 \quad \text{pour toute P-chaîne } C \in \mathcal{C}. \end{aligned}$$

Le système composé par (P) et (D) est TDI.

On peut remarquer que, pour tout vecteur w entier, il existe une solution entière de (P) qui correspond à une antiP-chaîne de poids maximal selon w . Par le théorème de Dilworth, on sait alors qu'il existe un ensemble $\mathcal{C}^x$ de P-chaînes qui couvre exactement w_i fois chaque élément i de V . Posons alors λ^x un vecteur entier indicé sur les P-chaînes, tel que $\lambda_C^x = 1$ si $C \in \mathcal{C}^x$ et 0 sinon. On peut alors remarquer que λ^x est solution de (D) car il vérifie les inégalités. De plus, la valeur objective de (D) pour λ^x est la cardinalité de $\mathcal{C}^x$.

Le théorème de Dilworth est : Soit un poset $(V, \prec)$ où chaque élément $i \in V$ est associé à un poids w_i entier. Le poids maximum $w(A)$ d'une antiP-chaîne de $(V, \prec)$ est égal à la cardinalité minimale d'un ensemble de P-chaînes de $(V, \prec)$ couvrant exactement $w(i)$ fois chaque élément i de V .

Donc, par le théorème de Dilworth, la cardinalité de $\mathcal{C}^x$ est égale à la solution optimale de (P) . Donc on a prouvé que pour tout vecteur w entier, chaque solution optimale entière x de (P) correspond à une solution entière de (D) de même valeur. Donc le système est TDI. Ce qui signifie que (P) a toutes ses solutions entières et que le problème PAM est équivalent à un PL. $\square$

Séparation des inégalités d P-chaînes

Considérons un ensemble C_0 d'inégalités de P-chaînes et le programme linéaire (P_0) contenant les inégalités C_0 et les inégalités triviales. Notons x^0 la solution optimale de (P_0) .

Lemma

Le problème de séparation des inégalités de P-chaînes pour x^0 est polynomial.

Le problème de séparation des contraintes de P-chaîne consiste à déterminer s'il existe ou non une inégalité de P-chaîne violée par x_0 et en produire une si c'est le cas. On associe la valeur x_i à chaque arc de G sortant de i . Pour chaque sommet i , on recherche l'arborescence des plus longs chemins issus de i : ceci peut être fait car G ne contient pas de circuit par l'algorithme de Bellman. Pour chaque sommet i , on retient le plus long chemin selon le poids du chemin $+ x_j^0$ où j est son autre extrémité. On retient ensuite le plus long chemin pour tout i . Cet algo polynomial fournit alors une P-chaîne de poids maximal selon x^0 . Si une telle P-chaîne a un poids ≥ 1 , alors on a déterminé une inégalité de P-chaîne violée par x^0 , sinon, toutes les P-chaînes correspondent à des inégalités non violées. Ainsi cet algo est polynomial et est équivalent au problème de séparation.

Conclusion pour les antichaînes

On sait résoudre la formulation PL des inégalités de P-chaînes en réitérant cet algo de séparation tant qu'il existe des contraintes violées, on a ainsi déterminé un algo de coupes.

Par le théorème de Grotschel-Lovasj-Schrijver, et par b), on sait qu'il est polynomial. De part le résultat de caractérisation, cela signifie que résoudre ce PL est une méthode polynomial pour déterminer une antichaîne maximale.

Et en pratique, c'est plus rapide que l'algo de Köning !
(Et cela marche pour des poids quelconque :)

1. Problème central $m \mid prec \mid C_{max}$
2. Dimensionnement de ressource en ordonnancement
3. Resource Constrained Project Scheduling Projet : RCPSP
4. Scheduling under uncertainty
5. Supply Chain par le problème de Production-Routing

See RCSP presentation pdf

1. Problème central $m \mid prec \mid C_{max}$
2. Dimensionnement de ressource en ordonnancement
3. Resource Constrained Project Scheduling Projet : RCPSP
4. Scheduling under uncertainty
5. Supply Chain par le problème de Production-Routing

See Robustesse presentation.pdf

1. Problème central $m \mid prec \mid C_{max}$
2. Dimensionnement de ressource en ordonnancement
3. Resource Constrained Project Scheduling Projet : RCPSP
4. Scheduling under uncertainty
5. Supply Chain par le problème de Production-Routing

See Supply Chain pdf